You said:

Where is my previous chat

ChatGPT said:

If you mean an earlier ChatGPT conversation, it should normally be in your **chat history/sidebar**.

Try:

1. Open the **left sidebar** in ChatGPT.
2. Look through **Recent chats**, or use **Search** to find a word or topic you remember.
3. Check **Archived chats** if you may have archived it.

I can also help recover what we were working on if you tell me a little about it. **Was it from today, an older chat, or do you remember the topic?**

You said:

```
# Creates a node for the Binary Search Tree

class Node:
   def __init__(self, key):
      self.key = key
      self.left = None
      self.right = None

# Initializes a node with a key and no children
```

ChatGPT said:

Yes. This code defines a **Node** for a Binary Search Tree (BST).

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
```

When you create a node:

```
node = Node(10)
```

it starts as:

```
    10
   /  \
None  None
```

- `self.key = key` stores the node's value, such as `10`.
- `self.left = None` means it initially has no left child.
- `self.right = None` means it initially has no right child.

So your comment "**Initializes a node with a key and no children**" is exactly right.

You said:

```
# Creates the Binary Search Tree

class BinarySearchTree:

    # Starts the tree with an empty root
    def __init__(self):
        self.root = None

    #Inserts a new key into the correct position
    def insert(self, key):
        if self.root is None:
            self.root = Node(key)
        else:
            current = self.root

            # Moves through the tree to find where they key belongs
            while True:
                if key < current.key:
                    if current.left is None:
                        current.left = Node(key)
                        break
                    else:
                        current= current.left
                else:
                    if current.right is None:
                        current.right = Node(key)
                        break
                    else:
                        current = current.right

    # Searches for a key and counts the nodes checked
```

```
def search(self, key):
    current = self.root
    comparisons = 0

    while current is not None:
        comparisons += 1

        if key == current.key:
            return True, comparisons

        elif key < current.key:
            current = current.left

        else:
            current = current.right

    return False, comparisons

# Visist the nodes in order from the smallest to largest
def inorder_traversal(self):
    result = []

    def traverse(node):
        if node is not None:
            traverse(node.left)
            result.append(node.key)
            traverse(node.right)

    traverse(self.root)
    return result

# Finds the maximum depth of the BST without recursion
def height(self):
    if self.root is None:
        return -1

    max_depth = 0
    stack = [(self.root, 0)]

    while stack:
        node, depth = stack.pop()
        max_depth = max(max_depth, depth)
```

```
            if node.left is not None:
                stack.append((node.left, depth + 1))

            if node.right is not None:
                stack.append((node.right, depth + 1))

        return max_depth

    # Finds the depth of every node without recursion
    def node_depths(self):
        depths = {}

        if self.root is None:
            return depths
        stack = [(self.root, 0)]

        while stack:
            node, depth = stack.pop()
            depths[node.key] = depth

            if node.left is not None:
                stack.append((node.left, depth + 1))

            if node.right is not None:
                stack.append((node.right, depth + 1))

        return depths
```

ChatGPT said:

Your `BinarySearchTree` class is structured correctly and works with the `Node` class you provided. Here's what each method is doing:

- **`__init__()`** — Creates an empty BST by setting `self.root = None`.
- **`insert(key)`** — Inserts a new key by starting at the root and moving left for smaller values or right for larger/equal values until an empty position is found.
- **`search(key)`** — Searches for a key and returns `(True, comparisons)` if found or `(False, comparisons)` if not found. The comparison count represents how many nodes were examined.
- **`inorder_traversal()`** — Uses recursive traversal in the order **left → root → right**, so the resulting list is sorted from smallest to largest.

- **`height()`** — Uses a stack instead of recursion to determine the maximum depth. With your definition, a single-node tree has height `0`, while an empty tree has height `-1`.
- **`node_depths()`** — Uses a stack to determine the depth of every node, where the root has depth `0`.

For example, inserting:

```
bst = BinarySearchTree()

for key in [50, 30, 70, 20, 40, 60, 80]:
    bst.insert(key)
```

produces:

```
        50
       /  \
     30    70
    / \    / \
   20 40  60 80
```

Then:

```
print(bst.inorder_traversal())
# [20, 30, 40, 50, 60, 70, 80]

print(bst.search(60))
# (True, 3)

print(bst.height())
# 2

print(bst.node_depths())
# {50: 0, 70: 1, 80: 2, 60: 2, 30: 1, 40: 2, 20: 2}
```

One small typo is in this comment:

```
# Visist the nodes in order from the smallest to largest
```

Change **`Visist`** to **`Visits`**.

Also, your `insert()` method puts **duplicate keys on the right**. That's perfectly valid if that's the duplicate-handling rule your assignment expects.

You said:

```
# Part 2: Ascending insertion order with 100 nodes

N = 100

ascending_bst = BinarySearchTree()

for key in range(1, N + 1):
    ascending_bst.insert(key)

print("Tree height:", ascending_bst.height())
```

ChatGPT said:

Yes, this correctly handles **Part 2: inserting 100 nodes in ascending order**.

Because you insert:

`1, 2, 3, 4, ..., 100`

every new value is greater than the previous value, so every node becomes a **right child**:

```
1
 \
  2
   \
    3
     \
      4
       \
        ...
         \
         100
```

This creates a completely **unbalanced (right-skewed) BST**.

With your `height()` method, the root has depth `0`, so the expected output is:

`Tree height: 99`

There are **100 nodes**, but the height is **99** because height is measured by the number of **edges** on the longest path from the root to a leaf.

So this part of your code is correct.

You said:

```
# Calculate the average node depth for ascending insertion

depths = ascending_bst.node_depths()

average_depth = sum(depths.values()) / len(depths)

print("Average node depth:", average_depth)
```

ChatGPT said:

Yes, this is correct.

For your ascending BST, the 100 nodes have depths:

`0, 1, 2, 3, ..., 99`

Your code:

```
depths = ascending_bst.node_depths()

average_depth = sum(depths.values()) / len(depths)

print("Average node depth:", average_depth)
```

calculates the mean of those depths. Since the average of `0` through `99` is **49.5**, your output should be:

```
Average node depth: 49.5
```

So for the ascending insertion case, you have:

- **Number of nodes:** 100
- **Tree height:** 99
- **Average node depth: 49.5**

This demonstrates why inserting already-sorted data into a regular BST is inefficient: it produces a highly unbalanced tree that behaves much like a linked list.

You said:

```
# Calculate average successful search comparisons

total_comparisons = 0

for key in range(1, N + 1):
```

```
    found, comparisons = ascending_bst.search(key)
    total_comparisons += comparisons

average_comparisons = total_comparisons / N

print("Average search comparisons:", average_comparisons)
```

ChatGPT said:

Yes, this is also correct.

Your loop searches for **every key from 1 through 100** and records how many nodes are checked for each successful search.

Because the ascending BST is completely right-skewed:

```
Search for 1   → 1 comparison
Search for 2   → 2 comparisons
Search for 3   → 3 comparisons
...
Search for 100 → 100 comparisons
```

So the average is:

$$\frac{1 + 2 + 3 + \cdots + 100}{100} = 50.5$$

Your output should therefore be:

```
Average search comparisons: 50.5
```

Notice the relationship with your previous result:

Average comparisons = Average node depth + 1

So:

```
49.5 + 1 = 50.5
```

The extra `1` occurs because a node at depth `0` still requires one comparison with the root.

You said:

```
# Test ascending insertion for different tree sizes

sizes = [100, 500, 1000, 2000]
```

```
ascending_results = []

for N in sizes:
    ascending_bst = BinarySearchTree()

    # Insert keys in ascending order
    for key in range(1, N + 1):
        ascending_bst.insert(key)

    # Calculate tree height
    tree_height = ascending_bst.height()

    # Caclulate avearge node depth
    depths = ascending_bst.node_depths()
    average_depth = sum(depths.values()) / N

    # Calculate average successful search commparisons
    total_comparisons = 0

    for key in range(1, N + 1):
        found, comparisons = ascending_bst.search(key)
        total_comparisons += comparisons

    average_comparisons = total_comparisons / N

    # Save the results
    ascending_results.append(
        (N, tree_height, average_depth, average_comparisons)
    )

    print(N, tree_height, average_depth, average_comparisons)
```

ChatGPT said:

Yes, this code correctly extends your ascending-insertion experiment to **100, 500, 1000, and 2000 nodes**.

Because ascending insertion always produces a completely right-skewed BST, you should get:

N	Tree Height	Average Node Depth	Average Search Comparisons
100	99	49.5	50.5

N	Tree Height	Average Node Depth	Average Search Comparisons
500	499	249.5	250.5
1000	999	499.5	500.5
2000	1999	999.5	1000.5

So your printed output should look like:

```
100 99 49.5 50.5
500 499 249.5 250.5
1000 999 499.5 500.5
2000 1999 999.5 1000.5
```

Your results follow three clear patterns:

```
Tree height                = N - 1
Average node depth         = (N - 1) / 2
Average search comparisons = (N + 1) / 2
```

As `N` increases, all three measurements grow **linearly with N**, or **O(N)**. This happens because ascending insertion creates the worst-case shape for an ordinary BST:

```
1
 \
  2
   \
    3
     \
      ...
       \
        N
```

One thing I'd clean up before submitting is the spelling in your comments:

```
# Calculate average node depth

# Calculate average successful search comparisons
```

You currently have `Caclulate`, `avearge`, and `commparisons`. The actual Python logic looks good.

You said:

```
# Test random insertion for different tree sizes

sizes = [100, 500, 1000, 2000]

random_results = []

for N in sizes:
    keys = list(range(1, N + 1))
    random.shuffle(keys)

    random_bst = BinarySearchTree()

    for key in keys:
        random_bst.insert(key)

    # Calculate tree height
    tree_height = random_bst.height()

    # Calculate average node depth
    depths = random_bst.node_depths()
    average_depth = sum(depths.values()) / N

    # Calculate average successful search comparisons
    total_comparisons = 0

    for key in range(1, N + 1):
        found, comparisons = random_bst.search(key)
        total_comparisons += comparisons

    average_comparisons = total_comparisons / N

    # Save results
    random_results.append(
        (N, tree_height, average_depth, average_comparisons)
    )

    print(N, tree_height, average_depth, average_comparisons)
```

ChatGPT said:

Yes, your random-insertion experiment is structured correctly. There is just **one important thing to check**: you need to import Python's `random` module before calling `random.shuffle()`.

Add this before the experiment if you haven't already:

```
import random
```

Then this part:

```
keys = list(range(1, N + 1))
random.shuffle(keys)
```

creates `[1, 2, ..., N]` and randomly rearranges it before insertion.

Unlike ascending insertion, the resulting BST should be **much more balanced**, something like:

```
          47
        /    \
      23      81
     /  \    /  \
   ...  ... ...  ...
```

Because the order is random, **your exact results will change each time you run the program**. That's expected.

You should see a major difference compared with ascending insertion:

N	Ascending Height	Random Height (typical)
100	99	much smaller
500	499	much smaller
1000	999	much smaller
2000	1999	much smaller

The same applies to average depth and search comparisons. Random insertion generally produces growth closer to **O(log N)** rather than the **O(N)** behavior of ascending insertion.

Also, this relationship should still hold:

```
average search comparisons = average node depth + 1
```

because finding a node at depth `d` requires checking `d + 1` nodes.

One improvement for your experiment

Since a single random shuffle can occasionally produce an unusually good or bad tree, a stronger experiment would run **several random trials for each N and average the results**. If your assignment only asks for one random insertion per size, though, your current code is appropriate.

And yes—your `random_results` list is set up correctly for comparing the random results against your `ascending_results` later.

You said:

I implemented my Binary Search Tree (BST) code and completed experiments using ascending, random, and balanced insertion orders for N = 100, 500, 1000, and 2000. I have saved my results in `ascending_results`, `balanced_results`, and `trial_results`, with 10 random trials for each tree size. Can you help me write simple Python code using Matplotlib to create two graphs? The first graph should compare tree height versus N for all three insertion strategies, using the average height across the 10 random trials. The second graph should compare the average number of successful search comparisons versus N for all three strategies, also averaging the 10 random trials. Both graphs should include labeled axes, titles, legends, and distinguishable lines. Please use my existing variables and include short `#` comments explaining what each part of the code does.

ChatGPT said:

Assuming each tuple in `ascending_results` and `balanced_results` is:

```
(N, height, average_depth, average_comparisons)
```

and `trial_results` contains the same type of tuple for each of your 10 random trials per `N`, you can use:

```
import matplotlib.pyplot as plt

# Get N values and results for ascending insertion
n_values = [result[0] for result in ascending_results]
ascending_heights = [result[1] for result in ascending_results]
ascending_comparisons = [result[3] for result in ascending_results]

# Get results for balanced insertion
```

```
balanced_heights = [result[1] for result in balanced_results]
balanced_comparisons = [result[3] for result in balanced_results]


# Calculate average random results for each N
random_heights = []
random_comparisons = []

for N in n_values:
    # Get the 10 trials that belong to this tree size
    trials = [result for result in trial_results if result[0] == N]

    # Average the height across the 10 trials
    average_height = sum(result[1] for result in trials) / len(trials)
    random_heights.append(average_height)

    # Average the search comparisons across the 10 trials
    average_search = sum(result[3] for result in trials) / len(trials)
    random_comparisons.append(average_search)


# ---------------- Graph 1: Tree Height ----------------

plt.figure(figsize=(8, 5))

plt.plot(n_values, ascending_heights, marker="o", label="Ascending")
plt.plot(n_values, random_heights, marker="s", label="Random")
plt.plot(n_values, balanced_heights, marker="^", label="Balanced")

plt.xlabel("Number of Nodes (N)")
plt.ylabel("Tree Height")
plt.title("Tree Height vs. Number of Nodes")
plt.legend()
plt.grid(True)

plt.show()


# ---------------- Graph 2: Search Comparisons ----------------

plt.figure(figsize=(8, 5))
```

```
plt.plot(n_values, ascending_comparisons, marker="o", label="Ascending")
plt.plot(n_values, random_comparisons, marker="s", label="Random")
plt.plot(n_values, balanced_comparisons, marker="^", label="Balanced")

plt.xlabel("Number of Nodes (N)")
plt.ylabel("Average Successful Search Comparisons")
plt.title("Average Search Comparisons vs. Number of Nodes")
plt.legend()
plt.grid(True)

plt.show()
```

The different markers (`o` , `s` , and `^`) make the three strategies distinguishable even without manually choosing colors.

One thing to verify is the structure of **`trial_results`** . The code above assumes all 40 random results are stored together as tuples beginning with `N` , such as:

```
(100, height, average_depth, average_comparisons)
```

If you paste the section where you created `trial_results` and `balanced_results` , I can make sure the graphing code matches your variables exactly.

You said:

I have already implemented my Binary Search Tree (BST) class in Python, including methods for insertion, searching, calculating tree height, and finding node depths. I also created a `balanced_order()` function. Can you help me write simple Python code to create three BSTs containing 1000 nodes each, using ascending, random, and balanced insertion orders? For random insertion, please use a fixed random seed so the results can be reproduced. I would also like to calculate and print the height of each tree to compare them. Please use my existing functions, keep the code simple, and include short `#` comments explaining what each section does.

ChatGPT said:

Assuming your existing class is named `BinarySearchTree` and your `balanced_order()` function accepts a list of keys, you can use:

```
import random
```

```
# Number of nodes in each BST
N = 1000


# ---------------- Ascending BST ----------------

ascending_bst = BinarySearchTree()

# Insert keys from 1 to 1000 in ascending order
for key in range(1, N + 1):
    ascending_bst.insert(key)

# Calculate the height
ascending_height = ascending_bst.height()


# ---------------- Random BST ----------------

# Use a fixed seed so the results are reproducible
random.seed(42)

# Create and shuffle the keys
random_keys = list(range(1, N + 1))
random.shuffle(random_keys)

random_bst = BinarySearchTree()

# Insert keys in random order
for key in random_keys:
    random_bst.insert(key)

# Calculate the height
random_height = random_bst.height()


# ---------------- Balanced BST ----------------

# Create the balanced insertion order
keys = list(range(1, N + 1))
balanced_keys = balanced_order(keys)

balanced_bst = BinarySearchTree()
```

```
# Insert keys in balanced order
for key in balanced_keys:
    balanced_bst.insert(key)

# Calculate the height
balanced_height = balanced_bst.height()


# ---------------- Compare Heights ----------------

print("Ascending BST height:", ascending_height)
print("Random BST height:", random_height)
print("Balanced BST height:", balanced_height)
```

You should expect the **ascending tree** to have a height of `999`. The **balanced tree** should have a height around `9` if your `balanced_order()` function produces a near-perfectly balanced insertion order. The random tree's exact height depends on the shuffled order, but using `random.seed(42)` means you'll get the same result each time you run it.

This gives you a straightforward demonstration of how **insertion order affects BST shape and height**.

You said:

ok

ChatGPT said:

Great. Your next comparison now has all three insertion strategies—**ascending, random, and balanced**—using the same 1000 keys, so the difference in tree height should be very clear.

You said:

```
# Part 3: Figure 3 - Node-Depth Distribution

import matplotlib.pyplot as plt

# Convert node depths into lists
ascending_values = list(ascending_depths.values())
random_values = list(random_depths.values())
balanced_values = list(balanced_depths.values())

# Create three histograms
```

```
fig, axes = plt.subplots(3, 1, figsize=(10, 12))

# Ascending BST
axes[0].hist(ascending_values, bins=range(1001))
axes[0].set_title("Ascending BST")
axes[0].set_xlabel("Node Depth")
axes[0].set_ylabel("Number of Nodes")

# Random BST
axes[1].hist(random_values, bins=range(22))
axes[1].set_title("Random BST")
axes[1].set_xlabel("Node Depth")
axes[1].set_ylabel("Number of Nodes")

# Balanced BST
axes[2].hist(balanced_values, bins=range(11))
axes[2].set_title("Balanced BST")
axes[2].set_xlabel("Node Depth")
axes[2].set_ylabel("Number of Nodes")

plt.tight_layout()
plt.show()
```

ChatGPT said:

Yes, your Figure 3 code is appropriate for comparing the **node-depth distributions** of the three 1000-node BSTs.

Just make sure you've calculated these variables first:

```
# Get the node depths for each BST
ascending_depths = ascending_bst.node_depths()
random_depths = random_bst.node_depths()
balanced_depths = balanced_bst.node_depths()
```

Then your histogram code will work.

One improvement I'd recommend is making the `bins` depend on the actual maximum depth rather than hard-coding `1001`, `22`, and `11`. That way your code still works if the random tree has a different height:

```
# Part 3: Figure 3 - Node-Depth Distribution

import matplotlib.pyplot as plt
```

```
# Get the node depths for each BST
ascending_depths = ascending_bst.node_depths()
random_depths = random_bst.node_depths()
balanced_depths = balanced_bst.node_depths()

# Convert node depths into lists
ascending_values = list(ascending_depths.values())
random_values = list(random_depths.values())
balanced_values = list(balanced_depths.values())

# Create three histograms
fig, axes = plt.subplots(3, 1, figsize=(10, 12))

# Ascending BST
axes[0].hist(ascending_values, bins=range(max(ascending_values) + 2))
axes[0].set_title("Ascending BST")
axes[0].set_xlabel("Node Depth")
axes[0].set_ylabel("Number of Nodes")

# Random BST
axes[1].hist(random_values, bins=range(max(random_values) + 2))
axes[1].set_title("Random BST")
axes[1].set_xlabel("Node Depth")
axes[1].set_ylabel("Number of Nodes")

# Balanced BST
axes[2].hist(balanced_values, bins=range(max(balanced_values) + 2))
axes[2].set_title("Balanced BST")
axes[2].set_xlabel("Node Depth")
axes[2].set_ylabel("Number of Nodes")

# Prevent labels from overlapping
plt.tight_layout()

# Display the figure
plt.show()
```

What you should see

The **ascending BST** should have exactly one node at every depth from `0` to `999`, because it is essentially a linked list.

The **random BST** should have most nodes concentrated around the middle depths, with relatively few nodes near the root or at the tree's maximum depth.

The **balanced BST** should show increasing numbers of nodes at each depth—approximately `1, 2, 4, 8, 16, ...` —until the final level. This demonstrates why the balanced BST provides much more efficient searches than the ascending BST.

You said:

can you help me and explain Extend your BST implementation to support Delete(key). Construct a BST using the following insertion sequence:
[50, 30, 70, 20, 40, 60, 80, 35, 45]
Perform these deletions in order:
1. Delete 20 (a leaf node).
2. Delete 40 (a node with two children at this point).
3. Delete 70 (a node with two children).
For each deletion, show the BST before and after the operation and explain how the tree
structure changes. When deleting a node with two children, use its inorder successor as the
replacement.
Investigation: Modify your deletion algorithm to use the inorder predecessor instead. Compare
the resulting tree structures after the same deletion sequence. Do the two strategies always
produce identical BSTs? Could their resulting structures affect future search performanc

ChatGPT said:

Absolutely. This part of the assignment is asking you to add a **delete operation** to your existing `BinarySearchTree` class, then compare two ways of deleting a node that has **two children**.

1. Understanding BST deletion

There are three possible situations when deleting a node:

1. **Leaf node** — it has no children, so simply remove it.
2. **One child** — remove the node and connect its parent directly to its child.
3. **Two children** — replace the node's key with another appropriate key, then delete that replacement key from its original location.

Your assignment first asks you to use the **inorder successor** for case 3.

The inorder successor is the **smallest key in the node's right subtree**.

For example, if deleting `40`:

```
    40
   /  \
  35   45
```

the inorder successor is `45`.

---

2. Add `delete()` to your BST class

You can put this method inside your existing `BinarySearchTree` class:

```
def delete(self, key):
    # Start at the root
    current = self.root
    parent = None

    # Find the node that contains the key
    while current is not None and current.key != key:
        parent = current

        if key < current.key:
            current = current.left
        else:
            current = current.right

    # Key was not found
    if current is None:
        return False

    # Case 1 and Case 2:
    # Node has zero or one child
    if current.left is None or current.right is None:

        if current.left is not None:
            child = current.left
        else:
            child = current.right
```

```
        # Deleting the root
        if parent is None:
            self.root = child

        # Connect parent to the node's child
        elif parent.left == current:
            parent.left = child
        else:
            parent.right = child

    # Case 3: Node has two children
    else:
        successor_parent = current
        successor = current.right

        # Find the smallest node in the right subtree
        while successor.left is not None:
            successor_parent = successor
            successor = successor.left

        # Replace current key with successor key
        current.key = successor.key

        # Remove successor from its original position
        if successor_parent.left == successor:
            successor_parent.left = successor.right
        else:
            successor_parent.right = successor.right

    return True
```

This uses the **inorder successor**, as your assignment specifies.

---

3. Construct the required BST

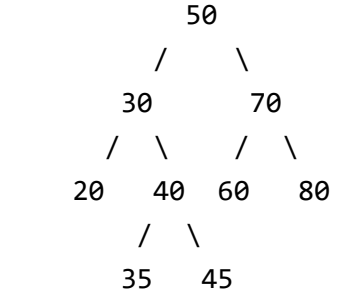
Your insertion sequence is:

```
keys = [50, 30, 70, 20, 40, 60, 80, 35, 45]

bst = BinarySearchTree()
```

```
for key in keys:
    bst.insert(key)
```

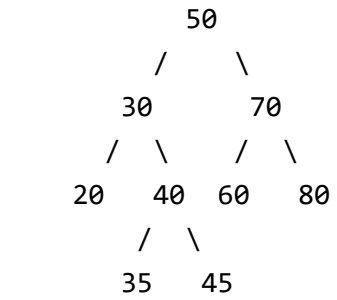
Before any deletion, your tree looks like this:



---

4. Delete 20 — leaf node

Before

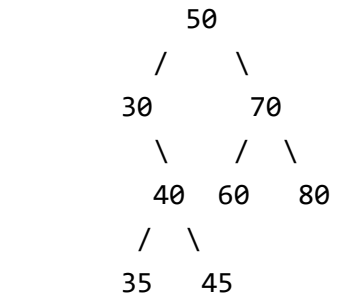


Run:

```
bst.delete(20)
```

`20` has no children, so this is the easiest case. We simply remove it.

After

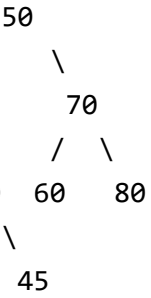


Explanation: `20` was a leaf node, so removing it does not require moving any other nodes.

5. Delete 40 — two children

Now the tree is:

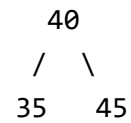
Before



Run:

```
bst.delete(40)
```

`40` has two children:



We need its **inorder successor**.

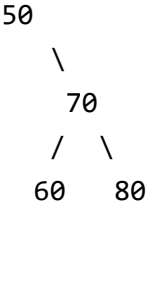
Look at the right subtree:

45

The smallest value there is `45`, so `45` is the inorder successor.

Replace `40` with `45`, and then remove the original `45`.

After



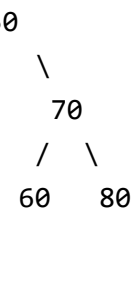
Explanation: `40` had two children, so its key was replaced by its inorder successor, `45`. The original `45` was then removed.

---

6. Delete 70 — two children

Our current tree is:

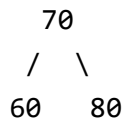
Before



Run:

```
bst.delete(70)
```

`70` also has two children:

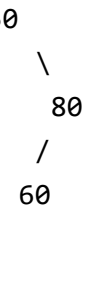


The inorder successor is the smallest value in the right subtree.

That is `80`.

Replace `70` with `80`, then delete the original `80`.

After



So after all three deletions, using the **inorder successor**, your final tree is:

```
      50
    /    \
  30      80
    \    /
    45  60
   /
  35
```

---

7. Investigation: use the inorder predecessor

Now your assignment asks you to change the algorithm.

Instead of using the **inorder successor**, use the **inorder predecessor**.

The difference is:

- **Successor:** smallest key in the **right subtree**
- **Predecessor:** largest key in the **left subtree**

For example:

```
   40
  /  \
 35   45
```

When deleting `40`:

- successor = `45`
- predecessor = `35`

So the resulting tree can be different.

You can make another deletion method:

```
def delete_predecessor(self, key):
    # Start at the root
    current = self.root
    parent = None

    # Find the node to delete
    while current is not None and current.key != key:
        parent = current
```

```
        if key < current.key:
            current = current.left
        else:
            current = current.right

    # Key was not found
    if current is None:
        return False

    # Node has zero or one child
    if current.left is None or current.right is None:

        if current.left is not None:
            child = current.left
        else:
            child = current.right

        if parent is None:
            self.root = child
        elif parent.left == current:
            parent.left = child
        else:
            parent.right = child

    # Node has two children
    else:
        predecessor_parent = current
        predecessor = current.left

        # Find largest key in left subtree
        while predecessor.right is not None:
            predecessor_parent = predecessor
            predecessor = predecessor.right

        # Replace current key with predecessor key
        current.key = predecessor.key

        # Remove predecessor from original location
        if predecessor_parent.right == predecessor:
            predecessor_parent.right = predecessor.left
        else:
```

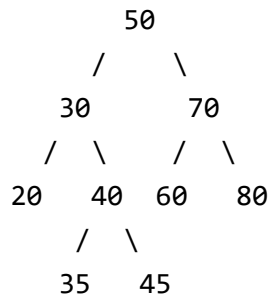
```
            predecessor_parent.left = predecessor.left

    return True
```

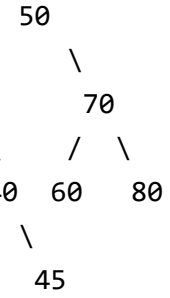
---

8. What happens with the predecessor strategy?

Start over with the original tree:



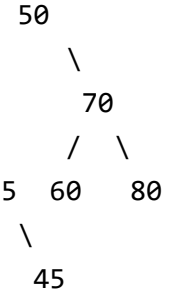
Delete `20` first. This doesn't change between the two strategies because `20` is a leaf:



Now delete `40`.

Its predecessor is `35`.

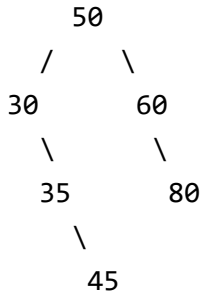
So you get:



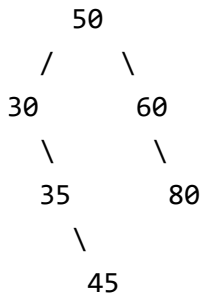
Next delete `70`.

Its predecessor is `60`.

You get:



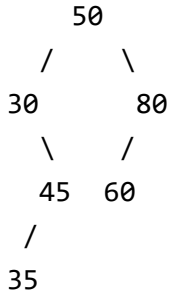
So the final tree using the **predecessor** is:



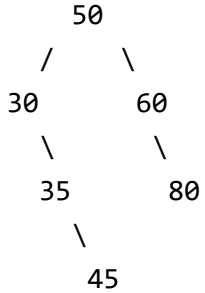
---

9. Compare the final trees

Using the **successor**:



Using the **predecessor**:



Both are **valid BSTs**, and both contain exactly the same remaining keys:

```
30, 35, 45, 50, 60, 80
```

But their structures are different.

Answer to the investigation questions

Do the two strategies always produce identical BSTs?

No. When deleting a node with two children, choosing the inorder successor versus the inorder predecessor can produce different tree structures. Both approaches preserve the BST ordering property, but they do not necessarily place the remaining nodes in the same positions.

Could the resulting structures affect future search performance?

Yes. Search performance depends on the **depth of the nodes** and the overall **height and shape of the BST**. If one deletion strategy results in a more balanced tree than the other, future searches may require fewer comparisons. Over many insertions and deletions, repeatedly choosing successors or predecessors can therefore influence the shape of the tree.

For this particular example, the differences aren't huge because the tree is small. But with a large BST and many deletions, the structural differences can become much more significant.

You said:

Can you help me combine my four answers from Part 2 of my CSCI 605 Binary Search Tree assignment into a summarized analysis? My professor wants a 2–3-page report covering Parts 2 and 3, so I don't want to answer each question separately. Instead, I want the answers combined into a few paragraphs that flow naturally."

1. Which insertion strategy produces the tallest BST? Explain why. Ascending insertion producest the tallest BST becasue the keys are inserted from smallest to largest. Since each new key is greater than the previous key, it is always inserted to the right. This causes the BST to grow into a straight chain instead of branching into left and right subtrees. In my experiment, when N = 2000, the ascending BST had a height of 1999, whiel the random BST had an average height of 23.8 and the balanced BST had a height of 10. This shows that ascending insertion produces the tallest tree because the nodes are not distributed evenly.

2. How does the height of the BST change as N increases for each strategy? Relate your observations to the theoretical time complexity of BST operations.

As N increases, the height of the BST increases differently depending on the insertion strategy. For ascending insertion, the height increases directly with N because every new node is added to the right, creating a chain. This resulsts in O(N) time commplexity for searching in the worst case. For random insertion, the height increases much more slowly because the nodes are distributed across different branches, although the tree is not guaranteed to be balanced. Random insertion generally has O(logN) expected search time. Balanced insertion produces the shortest trees because the keys are inserted in an order that keeps the left and right subtrees approximately balanced. This results in O(logN) search time. My experiments showed that when N increased from 100 to 2000, the ascending tree height increased from 99 to 1999, while the balanced tree height only increased from 6 to 10.

3. Compare the average successful search comparisons across the three strategies. How closely do these results correspond to the tree heights?
   The average number of successful search comparisons was highest for ascending insertion, followed by random insertion, and lowest for balanced insertion. This is because taller trees generally require more comparisons to find a key. In my experiment with N = 2000, ascending insertion required an average of 1000.5 comparisons, random insertion required 13.27, and balanced insertion required 9.98. The ascending BST required significantly more commparisons because the nodes formed a long chain. Random insertion required fewer comparisons because the tree had shorter paths, while balanced insertion required the fewest because its nodes were distributed more evenly. These results show that tree height is related to search performance, although the average depth of all nodes determines the exact average number of comparisons.

4. Is a BST constructed using random insertion necessarily balanced? Explain based on your experiments.
   No, a BST constructed using random insertion is not necessarily balanced. Random insertion places keys in a shuffed order, which usually creates a shorter tree than ascending insertion. However,

random insertion does not guarantee that the left and right subtrees will have similar heights. Some brances may still become longer than others. In my experiment with N = 1000, the random BST had an average height of 21.1 across 10 trialsm while the balanced BST had a height of 9. This shows that random insertion can produce a relatively efficient BST, but it does not gurantee the smallest possible height. Balanced insertion is more consistent because it intentionally selects middle keys to keep the tree height small."

ChatGPT said:

Yes. For a 2–3-page report covering both Parts 2 and 3, combining these observations into a continuous analysis will read much better than answering the questions individually. I'd keep your experimental numbers but remove repetition and connect the results to BST theory.

Here's a polished version that stays fairly simple and close to your original writing style:

Part 2 – BST Insertion Strategy Analysis

The experiments demonstrate that the order in which keys are inserted into a Binary Search Tree has a significant effect on the tree's structure and search performance. Of the three insertion strategies tested, ascending insertion consistently produced the tallest BST. Since the keys were inserted from smallest to largest, each new key was greater than the previous keys and was therefore placed in the right subtree. This caused the tree to grow as a straight chain rather than branching into left and right subtrees. As the number of nodes increased, the height of the ascending BST increased almost directly with N. For example, the height increased from 99 when N = 100 to 1999 when N = 2000. In comparison, at N = 2000, the random BST had an average height of only 23.8 across the trials, while the balanced BST had a height of 10.

The differences in tree height also demonstrate the relationship between insertion order and the theoretical time complexity of BST operations. In the ascending BST, the chain-like structure represents the worst case because searching may require moving through nearly every node in the tree. Therefore, search operations can have O(N) time complexity. Random insertion produced much shorter trees because the shuffled keys were distributed across different branches. Although random insertion does not guarantee a balanced tree, it generally produces expected search performance closer to O(log N). Balanced insertion produced the shortest and most consistent trees because the insertion order was intentionally chosen to keep the left and right subtrees approximately balanced. As a result, the balanced BST maintains a height that grows approximately logarithmically with N and supports O(log N) search performance. This difference was clear in the experiment, since the balanced tree height increased only from 6 at N = 100 to 10 at N = 2000.

The average number of successful search comparisons followed a similar pattern. Ascending insertion required the most comparisons, random insertion required substantially fewer, and balanced insertion required the fewest. At N = 2000, the ascending BST required an average of 1000.5 comparisons for a successful search, compared with 13.27 for random insertion and 9.98 for balanced insertion. The large number of comparisons in the ascending tree results from its chain-like structure, where many nodes are located far from the root. Random and balanced trees have shorter paths from the root to most nodes, which reduces the number of comparisons needed during a search. These results show that tree height is strongly related to search efficiency, although the exact average number of successful search comparisons depends on the average depth of all nodes rather than only the maximum height of the tree.

Finally, the experiments show that random insertion should not be considered the same as balanced insertion. Randomizing the insertion order usually prevents the extreme chain-like structure created by sorted input, but it does not guarantee that the left and right subtrees will have equal or similar heights. Some branches can still become longer than others depending on the random order of the keys. For example, at N = 1000, the random BST had an average height of 21.1 across 10 trials, while the balanced BST had a height of only 9. Therefore, random insertion can produce a relatively efficient BST, but its structure varies between trials and it does not guarantee the minimum possible height. Balanced insertion is more consistent because it deliberately selects keys in an order that keeps the tree approximately balanced. Overall, the results demonstrate that the structure of a BST is strongly influenced by insertion order and that maintaining a balanced structure can greatly improve search efficiency as the number of nodes increases.

This should work well as the **Part 2 analysis section** of your report. It preserves all four answers, but instead of sounding like four separate homework responses, it develops one argument: **insertion order → tree structure → height → search performance → random vs. truly balanced trees**.

For a 2–3-page total report, I would keep this section around this length and then make **Part 3 roughly comparable or slightly shorter**, especially if your figures/tables count toward the page limit.

You said:

how do you make an analysis for the questions i answered in part 3?
"1. How do the node-depth distributions differ among the three insertion strategies?
When I compared the node-depth distributions, I noticed that each insertion strategy produced a different result. For ascending insertion, the nodes were spread across depths 0 to 999, with only one node at each depth. This happened because the tree formed a straight chain instead of branching out. For random insertion, most of the nodes were grouped around the middle depths, and the deepest node was at depth 20. This shows that random insertion created more branches instead of one long chain. For balanced insertion, most of the nodes

were around depths 8 and 9, with a maximum depth of 9. From these results, I noticed that balanced insertion kept the tree much shorter, while ascending insertion created the deepest tree.

2. Why might two BSTs with similar heights have different average search performance?
Two BSTs can have similar heights but still have different average search performance because height only tells us how deep the tree goes, not where all the nodes are located. For example, one tree might have most of its nodes closer to the root, while another might have more nodes farther down. Since searching starts at the root, nodes that are closer require fewer comparisons to find. This means that even if two trees have the same height, the tree with a lower average node depth will usually have better search performance when all keys are searched equally often.

3. Explain the relationship between average node depth and average successful search comparisons. Can you express this relationship mathematically for a BST containing N distinct keys, assuming each key is searched exactly once?
From my results, I noticed that average node depth and average successful search comparisons are closely related. When searching for a key in a BST, the search starts at the root and makes a comparison at each node until the key is found. Since the root starts at depth 0 but still needs one comparison, the number of comparisons is always the node's depth plus 1. This means that the average number of successful search comparisons is equal to the average node depth plus 1, which can be written as C_avg = D_avg + 1 when each key is searched exactly once. For example, in my experiment with 100 nodes, the balanced BST had an average node depth of 4.8 and an average of 5.8 comparisons. This matched the formula and helped me understand how node depth affects search performance.

ChatGPT said:

For Part 3, I would approach the analysis the same way as Part 2: **don't present three separate answers**. Instead, connect them into one discussion about how **node-depth distribution affects search performance**.

Your three answers already form a logical sequence:

insertion strategy → node-depth distribution → average depth → search comparisons

Here is a version you could use in your report:

Part 3 – Node-Depth and Search Performance Analysis

The node-depth distributions provide a clearer view of how each insertion strategy affects the structure and search performance of a BST. For the 1000-node trees, ascending insertion produced nodes at every depth from 0 to 999, with only one node at each depth. This occurred because inserting the keys in increasing order created a straight chain rather than allowing the tree to branch. Random insertion produced a much different distribution, with most nodes grouped around the middle depths and a maximum depth of 20. This indicates that random insertion allowed the nodes to spread across multiple branches, although the tree was not perfectly balanced. Balanced insertion produced the most compact distribution, with most nodes located around depths 8 and 9 and a maximum depth of 9. Overall, the depth distributions show that balanced insertion keeps nodes much closer to the root, while ascending insertion places many nodes increasingly far from the root.

These distributions are important because tree height alone does not completely describe search performance. Two BSTs can have similar heights but different average search times because height only measures the maximum depth of the tree. It does not show how the remaining nodes are distributed. For example, two trees could have the same height, but one might contain only a few nodes near its maximum depth while the other contains many nodes at deeper levels. Since every search begins at the root, a tree with more nodes closer to the root will generally require fewer comparisons for successful searches. Therefore, average node depth provides a better measurement of overall search performance than height alone when every key is searched equally often.

The experimental results also demonstrate a direct relationship between average node depth and the average number of successful search comparisons. A node at depth 0, which is the root, requires one comparison to find. A node at depth 1 requires two comparisons, and this pattern continues throughout the tree. Therefore, for any node, the number of successful search comparisons is equal to its depth plus one. When each of the N distinct keys is searched exactly once, this relationship also applies to the averages, so the average number of successful search comparisons can be expressed as **C_avg = D_avg + 1**, where C_avg represents the average number of comparisons and D_avg represents the average node depth. For example, in the experiment with 100 nodes, the balanced BST had an average node depth of 4.8 and required an average of 5.8 successful search comparisons, which agrees with this relationship. These results demonstrate that insertion order affects not only the maximum height of a BST but also the distribution of nodes throughout the tree, which directly influences overall search efficiency.

This also connects nicely with your **Part 2 analysis**. Part 2 establishes that balanced trees have lower heights and require fewer comparisons, while Part 3 explains **why** by examining where the nodes are actually located within each tree.

One small point worth emphasizing in your report: **height measures the worst/deepest path, while average node depth better explains average successful-search performance.** That's probably the most important analytical conclusion from Part 3.

You said:

For an analysis do i need to add anyting?

ChatGPT said:

Your Part 3 analysis already has the main things an analysis should include. You **do not need to add much more**, especially since your professor wants Parts 2 and 3 together in only 2–3 pages.

A strong analysis should do four things, and yours does all four:

1. **Describe what happened** — You explain the different depth distributions for ascending, random, and balanced insertion.
2. **Use your experimental results as evidence** — You include specific values such as maximum depths of `999`, `20`, and `9`.
3. **Explain why the results happened** — You connect insertion order to the shape of the BST.
4. **Connect the results to the concept/theory** — You explain why average depth affects search comparisons and give the relationship $C_{avg} = D_{avg} + 1$.

The one thing I might add is a **short concluding comparison** connecting Parts 2 and 3. Something like:

> Overall, the results from Parts 2 and 3 show that insertion order has a major effect on BST structure and efficiency. Ascending insertion produced the least efficient structure because it created a chain with large height and node depths. Random insertion performed much better but did not guarantee a balanced structure. Balanced insertion consistently produced the smallest height and kept nodes closer to the root, resulting in fewer successful search comparisons. These experiments demonstrate that maintaining a balanced BST becomes increasingly important as the number of nodes grows.

That would give your analysis a clear ending without adding unnecessary repetition.

Also, since you have **graphs from your experiments**, refer to them directly in the report—for example, *"As shown in Figure 3, the ascending BST has nodes distributed across depths 0 through 999…"* That makes it feel more like an experimental analysis rather than just answers to homework questions.

You said:

Can you review my analysis and tell me whether I answered all seven questions from Parts 2 and 3?

ChatGPT said:

Yes. Based on the **Part 2 and Part 3 analyses you shared above**, you answered all seven questions. Here's how they map to your combined report.

Question	Covered?	Where your analysis addresses it
Part 2, Q1: Which insertion strategy produces the tallest BST? Why?	✅	You identify **ascending insertion** and explain that inserting increasing keys creates a right-skewed chain.
Part 2, Q2: How does height change as N increases? Relate to theoretical complexity.	✅	You compare ascending, random, and balanced growth and connect them to **O(N)** and expected/**O(log N)** search behavior.
Part 2, Q3: Compare average successful search comparisons. How do they relate to height?	✅	You compare `1000.5`, `13.27`, and `9.98` at N = 2000 and explain the relationship between height/depth and comparisons.
Part 2, Q4: Is a randomly constructed BST necessarily balanced?	✅	You clearly say **no** and support it with your N = 1000 results: random average height `21.1` versus balanced height `9`.
Part 3, Q1: How do node-depth distributions differ?	✅	You explain the chain-like ascending distribution, middle-depth concentration for random insertion, and compact distribution for balanced insertion.
Part 3, Q2: Why can BSTs with similar heights have different average search performance?	✅	You correctly explain that **height only measures maximum depth**, while the distribution/average depth of all nodes affects average searches.
Part 3, Q3: Relationship between average node depth and successful search comparisons? Express mathematically.	✅	You give the correct relationship **C_avg = D_avg + 1** and explain why it works.

What is especially strong

Your report goes beyond simply stating the experimental results. You consistently follow the pattern:

result → explanation → connection to BST performance

For example, you don't just say the ascending tree has height 1999—you explain **why** it reaches 1999 and how that leads to **O(N)** worst-case search behavior. That's analysis rather than just reporting data.

Your Part 3 discussion also adds an important distinction: **tree height and average node depth are not the same measurement**. Height describes the deepest node, while average node depth better predicts the average number of comparisons when all keys are searched equally often.

A few improvements before submitting

I would make only minor changes.

First, refer to your figures directly where appropriate. For example:

> "As shown in Figure 3, the ascending BST has one node at each depth from 0 through 999, while the random and balanced BSTs have nodes concentrated at much smaller depths."

Second, make sure the terminology is consistent. Use **"average successful search comparisons"** rather than switching between "search time," "average search," and "comparisons" when discussing your measured experimental value. Search time and comparison count are related, but your experiment specifically measured comparisons.

Third, your statement that random insertion generally gives **O(log N) expected search performance** is appropriate, but keep the word **expected**. A randomly constructed BST is not guaranteed to be balanced or have logarithmic height in every individual trial.

Finally, end Parts 2 and 3 with the short overall conclusion we discussed. That will make the report feel like one analysis rather than seven answers joined together.

So yes—you have answered all seven questions. I don't see a Part 2 or Part 3 question from the material you provided that is missing from your analysis.

You said:

How do I upload this chat as a PDF for my professor to view or access?

ChatGPT said:

Yes. If your professor wants a **PDF copy of this ChatGPT conversation**, the easiest method is to use your browser's Print → PDF feature.

On your computer:

1. Open this conversation and scroll through it to make sure the parts you want included have loaded.
2. Press **Ctrl + P** on Windows or **Cmd + P** on Mac.
3. In the print window, choose **Save as PDF** as the destination/printer.
4. Check the preview to make sure your code and responses aren't cut off.
5. Click **Save** and give it a clear filename, such as `CSCI_605_ChatGPT_Conversation.pdf`.
6. Upload that PDF to Canvas, Blackboard, Moodle, or wherever your professor requested it.

If your professor specifically wants **evidence of your use of ChatGPT**, I would keep the conversation intact rather than copying only selected answers into another document. That makes it clearer which prompts you wrote and which responses came from ChatGPT.

If you'd like, I can also help you **organize this conversation into a clean AI-use appendix** for your CSCI 605 submission, while keeping your prompts and my responses clearly identified.